TrustVerify

Mini Project I Report

Information Security

Student: Yiğit Özdemir

Student Number: 220208027

Student: Berkay Yurttaş

Student Number: 220208042

GitHub Repository: [https://github.com/yigitozdmr/trustverify-cli]

Presentation Video: [https://youtu.be/Vkrn7nIV8rM]

University: OSTİM Technical University | 2026

Introduction

This project presents TrustVerify, a Python-based Command Line Interface (CLI) tool developed to verify the integrity and authenticity of digital files by using SHA-256 hashing and RSA digital signatures. The system follows a sender–receiver model. First, the sender generates a manifest file, metadata.json, which stores the SHA-256 hash values of files in a target directory. Then, the sender signs this manifest using a private RSA key. On the receiver side, the signature is verified with the corresponding public key, and the files are checked against the stored hash values.

The project demonstrates how cryptographic mechanisms can be used together to protect digital content against tampering and impersonation. In this way, the system addresses three essential information security properties: integrity, authenticity, and non-repudiation.

1. Why Hashing Alone Is Not Enough to Prove Identity

Cryptographic hash functions such as SHA-256 are highly effective for ensuring data integrity. A hash function converts input data into a fixed-length digest, and even a very small modification in the original file produces a completely different hash value. For this reason, hashing is widely used to detect unauthorized changes, accidental corruption, or incomplete transmission.

In this project, the hash values stored in metadata.json make it possible to determine whether any file has been modified. If the newly calculated hash of a file differs from the stored value, the system detects that the file has been altered.

However, hashing alone cannot prove the identity of the sender. The reason is that a hash is not tied to a specific person or source. Anyone, including an attacker, can generate the SHA-256 hash of any file. As a result, if an attacker intercepts a file, changes its content, and computes a new valid hash for the modified version, the receiver may still see a matching file-hash pair and incorrectly assume that the data is trustworthy.

This weakness can be explained through a Man-in-the-Middle (MitM) scenario. An attacker positioned between the sender and the receiver can replace both the original file and its hash with altered versions. Since the modified file still matches the new hash, the receiver can only verify consistency, not legitimacy. Therefore, hashing guarantees integrity, but by itself it does not provide authenticity or proof of identity.

2. How the Private/Public Key Relationship Ensures Non-Repudiation

The limitation of hashing is addressed by asymmetric cryptography, which uses a mathematically related pair of keys: a private key and a public key. The private key is kept secret by the sender, while the public key is shared with others. A message or hash signed with the private key can only be verified using the matching public key.

In the TrustVerify system, the sender signs the hash of metadata.json with the private key. This creates a digital signature that is cryptographically linked to the sender. The receiver then uses the sender's public key to verify the signature. If the verification is successful, two important conclusions can be made: first, the manifest has not been modified after signing; second, the signature could only have been produced by the holder of the corresponding private key.

This mechanism provides authenticity, because it connects the data to the sender's identity, and it also supports integrity, because any modification invalidates the signature. Most importantly, it ensures non-repudiation. Since only the sender should possess the private key, the sender cannot later deny having signed the data, assuming the private key has remained secure.

Unlike a simple hash value, which can be generated by anyone, a digital signature is uniquely associated with the legitimate key owner. This makes asymmetric cryptography essential for trustworthy and accountable digital communication.

Conclusion

The TrustVerify project demonstrates that SHA-256 hashing alone is not sufficient for secure file verification. Although hashing is effective for detecting changes and ensuring integrity, it cannot prove the identity of the sender.

By combining hashing with RSA digital signatures, the system provides a stronger security model that supports integrity, authenticity, and non-repudiation at the same time. As a result, the receiver can verify not only that the files have not been altered, but also that they were produced and signed by a legitimate and accountable source.